Documentação Técnica Plugin Matrox Monarch Control para Q-SYS

v2.0.2

Daniel Lorenzo Silveira Alves
daniel.lorenzo0610@gmail.com

05 de março de 2026

Introdução 3
Tutorial de Instalação e Utilização 4
Informações do Plugin 5
- **Metadados** 5
- **Identificação** 5

Arquitetura e Estrutura 5
Configurações e Propriedades 6
Interface e Controles 6
- **Leiaute da Interface** 7
- **Feedback Visual e Estados de Operação** 7
  - **Comportamento dos Botões de Ação (ENC1/ENC2)** 7
  - **Semáforo de Conectividade (HTTP STATUS)** 7

Lógica de Comunicação com o Matrox 8
- **Comandos Principais** 8
- **Fluxo de Polling** 8

Tratamento de Erros e Repetições 9
- **Mecanismo de Retry e Feedback Dinâmico** 9
- **Mensagens no Log** 9

Etapas de Execução 10
Notas de Implementação 10
Recomendações 10
Licença e Créditos 10
Contribuições 10
Agradecimentos 11
ANEXO I – Código Fonte Completo 11

Introdução

Este documento descreve a especificação e as funcionalidades do *plugin* Matrox Monarch Control, destinado à integração de software entre o codificador de vídeo Matrox Monarch HDX e a plataforma Q-SYS.

Desenvolvido em linguagem de programação Lua, o *plugin* possibilita integrações de controle e monitoramento de dois canais de codificação de vídeo (ENC1 e ENC2), incluindo ações de iniciar e parar gravações/streamings, visualizar status de cada *encoder* e gerenciar a conectividade com o dispositivo.

O *plugin* foi desenvolvido na versão LTS **9.4.8** do software Q-SYS Designer.

Tutorial de Instalação e Utilização

Figura 1: Clique na imagem para acessar o vídeo de demonstração no YouTube.

Informações do Plugin

Metadados

- Nome: User Matrox Monarch Control
- Versão: 2.0.2
- Autor: Daniel Lorenzo Silveira Alves
- Idealizador: Bruno Mariani de Melo
- Suporte: Equipe da Seção de Soluções Integradas de Áudio e Vídeo do STJ
- Fabricante do Equipamento: Matrox
- Modelo Compatível: Monarch HDX
- Descrição: Plugin desenvolvido para o controle de gravação/*streamning* e monitoramento de dois *encoders* do dispositivo Matrox Monarch HDX

Identificação

Id: devbydaniellorenzosavid.stj.matroxmonarchcontrol

DisplayName: Matrox Dual Encoder Control

Arquitetura e Estrutura

O plugin é dividido em sete blocos principais:

1. Estrutura e metadados;
2. Definição de propriedades configuráveis;
3. Definição de controles;
4. Layout e interface visual;
5. Funções auxiliares e utilitárias;
6. Comunicação HTTP com o Matrox;
7. Loop de *Polling* e *callbacks* dos controles.

Configurações e Propriedades

O plugin permite configurar os seguintes parâmetros:

Propriedade	Tipo	Descrição
HostIP	String	Endereço IP do Matrox
MatroxPort	String	Porta HTTPS utilizada (padrão: 443)
Username	String	Usuário de autenticação
Password	String	Senha (suporta caracteres especiais)
PollingInterval	Inteiro	Intervalo de atualização (1 a 30 segundos)
MaxRetries	Inteiro	Máximo de tentativas antes do *timeout*

Tabela 1: Propriedades configuráveis do plugin.

Interface e Controles

O plugin define dois grupos principais de controles, um para cada *encoder* (ENC1 e ENC2).

Cada grupo contém:

- Botão de Start/Stop (toggle)
- Indicador de modo
- Indicador de estado (“*Ready*") para disponível a iniciar gravação/*streaming* e "*On*" para gravação/*streaming* em andamento
- Indicador de status da solicitação HTTP

Leiaute da Interface

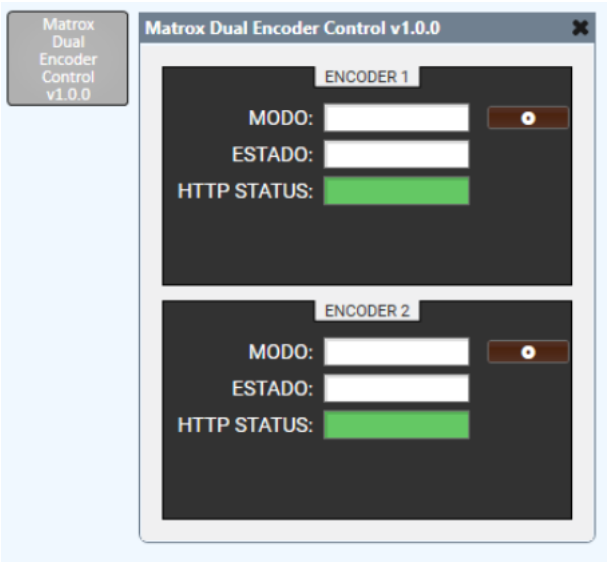

Figura 2: Leiaute geral do plugin no Q-SYS Designer.

Feedback Visual e Estados de Operação

A interface do plugin foi desenhada para fornecer *feedback* imediato ao operador sobre o modo de configuração do Matrox Monarch HDX e a integridade da comunicação. O comportamento visual dos botões e campos de status é dinâmico e segmentado conforme descrito abaixo:

Comportamento dos Botões de Ação (ENC1/ENC2)

O botão principal de cada encoder altera a sua "personalidade" (ícone e cor) dependendo se o dispositivo está configurado para gravação ou transmissão.

Modo Detectado	Estado	Ícone Visual	Cor do Botão
RECORD	**IDLE (Parado)**	Record (Círculo)	Marrom (#A0522D)
RECORD	**ACTIVE (Ligado)**	Stop (Quadrado)	Vermelho - Laranja (Piscando) (#FF4500)
RTMP / RTSP	**IDLE (Parado)**	RSS (Antena)	**Azul Escuro** (#1E90FF)
RTMP / RTSP	**ACTIVE (Ligado)**	Stop (Quadrado)	**Azul Claro** (Piscando) (#00BFFF)

Nota: O estado "Piscando" (Blinking) ocorre em intervalos de 0.5s para indicar atividade de processamento no hardware.

Semáforo de Conectividade (HTTP STATUS)

Para facilitar o diagnóstico de rede sem a necessidade de consultar os *logs*, o campo **HTTP STATUS** opera sob uma lógica de semáforo:

- **VERDE (OK):** Indica que a última requisição ao Matrox foi bem-sucedida (Código HTTP 200).
- **AMARELO (RETRY):** Indica uma falha temporária de comunicação. O plugin está a tentar restabelecer o contato (até ao limite definido em *Max Retries*). O texto exibirá RETRY... (x/n).
- **VERMELHO (OFFLINE/ERROR):** Indica que o limite de tentativas foi excedido ou que as credenciais estão incorretas. O botão de ação assumirá um ícone de alerta (!) e a cor cinza, indicando que o controle está desabilitado.

Lógica de Comunicação com o Matrox

A comunicação ocorre via protocolo **HTTPS GET**, utilizando autenticação embutida no URL:

https://USERNAME:PASSWORD@HOST:PORT/Monarch/syncconnect/sdk.aspx?command=<CMD>

Comandos Principais

- GetStatus Consulta o estado dos *encoders*
- StartEncoder1 / StopEncoder1 Controle do Encoder 1
- StartEncoder2 / StopEncoder2 Controle do Encoder 2

Fluxo de Polling

O plugin executa uma função de *Polling* a cada **PollingInterval** segundos, consultando o *status* e atualizando a interface em tempo real.

Tratamento de Erros e Repetições

O plugin conta com um sistema de resiliência que evita alarmes falsos em caso de oscilações momentâneas de rede, utilizando um feedback visual de três estágios.

Mecanismo de Retry e Feedback Dinâmico

O plugin monitora a resposta de cada requisição HTTP. Caso ocorra uma falha (*timeout*, erro de rede ou código diferente de 200), o mecanismo de **Retry** é ativado:

1. **Estado de Sucesso (Verde):** Quando a resposta é `200 OK`, o campo **HTTP STATUS** exibe "OK" na cor **Verde (#00FF00)**.
2. **Estado de Recuperação (Amarelo):** Em caso de falha, o campo muda para **Amarelo/Dourado (#FFC000)** e o texto indica o progresso das tentativas: `RETRY... (X/N)`, onde X é a tentativa atual e N é o máximo definido na propriedade *MaxRetries*. Durante este estágio, o plugin tenta manter a última informação conhecida na interface.
3. **Estado de Falha Crítica (TIMEOUT / Vermelho):** Se o contador atingir o limite de *MaxRetries* sem sucesso:
    - O campo **HTTP STATUS** muda para **Vermelho (#FF0000)** com a mensagem `TIMEOUT!` ou `OFFLINE`.
    - Os botões de controle (**ENC1/ENC2**) perdem suas cores de operação, assumindo a cor **Cinza (#808080)**.
    - O ícone dos botões é alterado para o símbolo de alerta (`!`), indicando que o controle está inoperante.
    - Qualquer animação de "pisca" (Blinking) é interrompida imediatamente.

Mensagens no Log

O plugin registra eventos importantes no *Debug Output* do Q-SYS para facilitar a manutenção técnica. As mensagens padronizadas são:

- **Sucesso após comando:** "Comando de controle Matrox OK. Forçando atualização de status."

- **Aviso de falha parcial:** "Falha/Retry Matrox. Tentativas: X. Code=YYY. Erro=[Descrição]" (Onde YYY é o código HTTP retornado).
- Erro de execução: "Comando Matrox falhou. Resposta: [Conteúdo da Resposta]"
- **Aviso de Timeout:** "TIMEOUT! Sem resposta há N segundos." (O tempo N é calculado com base no intervalo de polling e número de tentativas).

Etapas de Execução

1. O plugin é carregado e inicializa as propriedades.
2. Inicia o *polling* contínuo de *status*.
3. O usuário interage com os botões de controle na *UCI*.
4. O *plugin* envia comandos HTTPS para o Matrox.
5. O retorno HTTP é interpretado e reflete na interface.

Notas de Implementação

- Os ícones utilizados são codificados nos próprios arquivos de ícones da Q-SYS
- (ícones de gravação, stop, alerta, refresh).
- O módulo JSON é requerido localmente (require("json")).
- O uso de variáveis globais (_G) facilita o compartilhamento de estado entre funções de *polling* e resposta.

Recomendações

- Certifique-se de que o endereço IP e as credenciais de usuário do Matrox estejam corretos.
- O Matrox Monarch HDX deve estar acessível na rede via HTTPS.
- Para reduzir carga de rede e no Matrox, defina o PollingInterval acima de 10 segundos.

Licença e Créditos

Este plugin foi desenvolvido por **Daniel Lorenzo Silveira Alves** como parte de suas atividades de estágio/colaboração para o **Superior Tribunal de Justiça (STJ)** na Seção de **Soluções Integradas de Áudio e Vídeo (SAVID)**.

Para incentivar a colaboração e a reutilização por outras instituições e pela comunidade de integradores, este código está licenciado sob a **Licença MIT (Massachusetts Institute of Technology)**.

Contribuições

Propostas, sugestões e correções de bugs para a evolução do plugin podem ser enviadas por meio de **pull requests** e **issues** no repositório oficial do projeto: https://github.com/daniel-lorenzo-silveira/Q-SYS-Matrox-Monarch-Rec-Control-HDX

Agradecimentos

Este projeto foi desenvolvido com a valiosa colaboração e suporte técnico da Equipe SAVID, que forneceu o ambiente de testes e o conhecimento especializado necessário para a integração.

Um agradecimento especial ao meu supervisor, Bruno Mariani de Melo, cuja visão e idealização do projeto tornaram esta solução de controle para o Matrox Monarch HDX possível.

ANEXO I – Código Fonte Completo

--********** --* Agradecimentos e Créditos -- ********** -- Este plugin foi desenvolvido por Daniel Lorenzo Silveira Alves (Estagiário) -- Idealização do Projeto: Bruno Mariani de Melo -- Colaboração e Suporte Técnico: Equipe SAVID STJ

--********** -- 1. ESTRUTURA E METADADOS DO PLUGIN (Oficial) -- **********

PluginInfo = { Name = "User Matrox Monarch Control", Version = "2.0.2", Id = "devbydaniellorenzosavid.stj.matroxmonarchcontrol", DisplayName = "Matrox Dual Encoder Control", Author = "Daniel Lorenzo Silveira Alves daniel.lorenzo0610@gmail.com", Description = "Plugin desenvolvido para o controle de gravação, streaming e monitoramento dos dois encoders do dispositivo Matrox Monarch HDX", Manufacturer = "Matrox", Model = "Monarch HDX"

}

-- Define a cor do plugin object no design function GetColor(props) return { 102, 102, 102 } end

-- O nome que será exibido inicialmente function GetPrettyName(props) return "Matrox Dual Encoder Control v" .. PluginInfo.Version end

-- Define as configurações de propriedades do plugin function GetProperties() local props = {} table.insert(props, {Name = "HostIP", Type = "string", Value = "172.16.16.46", Display = "IP do Matrox", Group = "Configuração"}) table.insert(props, {Name = "MatroxPort", Type = "string", Value = "443", Display = "Porta HTTPS", Group = "Configuração"}) table.insert(props, {Name = "Username", Type = "string", Value = "admin", Display = "Usuário", Group = "Configuração"}) table.insert(props, {Name = "Password", Type = "string", Value = "Savid@Gravacao", Display = "Senha", Group = "Configuração"}) table.insert(props, {Name = "PollingInterval", Type = "integer", Min = 0 , Max = 30, Value = 10 ,Display = "Intervalo de Polling (s)", Group = "Polling"}) table.insert(props, {Name = "MaxRetries", Type = "integer", Min = 0 , Max = 10, Value = 4 , Display = "Máximo de Tentativas", Group = "Polling"}) return props end

-- Define os Controls usados no plugin function GetControls(props) local ctrls = {}

```
-- ENCODER 1 (GRAVAÇÃO)
table.insert(ctrls, {
    Name = "ENC1_StartButton",
    ControlType = "Button",
    ButtonType = "Toggle", -- Usamos Toggle para manter o estado (Rec/Stop)
    Display = "Ação ENC1 (Start/Stop)",
    UserPin = true,
```

```
    PinStyle                        =                        "Input",
    Icon                          =                         "Record"
})
table.insert(ctrls,  {Name  =  "ENC1_Mode",  ControlType  =  "Text",
Display          =          "Status          ENC1:          MODO"})
table.insert(ctrls,  {Name  =  "ENC1_State",  ControlType  =  "Text",
Display         =         "Status         ENC1:         ESTADO"})
table.insert(ctrls, {Name = "ENC1_HTTPStatus", ControlType = "Text",
Display          =          "Status          ENC1:          HTTP"})

--                            ENCODER                            2
table.insert(ctrls,                                              {
    Name                  =                   "ENC2_StartButton",
    ControlType                     =                    "Button",
    ButtonType                      =                    "Toggle",
    Display        =        "Ação        ENC2        (Start/Stop)",
    UserPin                         =                       true,
    PinStyle                        =                    "Input",
    Icon                          =                         "Record"
})
table.insert(ctrls,  {Name  =  "ENC2_Mode"  ,  ControlType  =  "Text",
Display          =          "Status          ENC2:          MODO"})
table.insert(ctrls,  {Name  =  "ENC2_State",  ControlType  =  "Text",
Display         =         "Status         ENC2:         ESTADO"})
table.insert(ctrls, {Name = "ENC2_HTTPStatus", ControlType = "Text",
Display          =          "Status          ENC2:          HTTP"})

return                                                          ctrls


end

-- Leiaute de controles e gráficos para a interface de usuário exibida do plugin function
GetControlLayout(props) local layout = {} local graphics = {}

local                 START_X                 =                 10
local                 START_Y                 =                 10
local                 SPACING                 =                 25
local               GROUP_WIDTH                =               280
local               GROUP_HEIGHT                =              150
local         X_LABEL         =         START_X         +         5
local         X_VALUE         =         START_X         +       110
local         X_BUTTON         =         START_X         +       220

--        GRÁFICO:        BOX        para        ENCODER        1
table.insert(graphics,                                           {
    Type                        =                       "GroupBox",
    Text                =                "ENCODER                1",
    Fill    =    {50,    50,    50},    --    Cor    de    fundo    escura
```

```
    StrokeWidth                          =                        1,
    Position            =            {START_X,            START_Y},
    Size          =          {GROUP_WIDTH,          GROUP_HEIGHT}
})

--        GRÁFICO:        BOX        para        ENCODER        2
table.insert(graphics,                                          {
    Type                      =                       "GroupBox",
    Text               =               "ENCODER               2",
    Fill            =            {50,            50,            50},
    StrokeWidth                          =                        1,
    Position   =   {START_X,   START_Y   +   GROUP_HEIGHT   +   10},
    Size          =          {GROUP_WIDTH,          GROUP_HEIGHT}
})

-----------------------------------------------------------
--       CONTROLES       DO       ENCODER       1       (GRAVAÇÃO)
-----------------------------------------------------------
local         Y1         =         START_Y         +         25

--                        Botão                        START/STOP
layout["ENC1_StartButton"]                  =                  {
    PrettyName          =          "BOTAO          INICIAR/PARAR",
    Style                       =                       "Button",
    Position             =             {X_BUTTON,             Y1},
    Size                =                {60,                20},
    Color   =   {160,   82,   45}   --   Cor   inicial   (IDLE)
}

--                Status                MODO                (LABEL)
table.insert(graphics, { Type = "Label", Text = "MODO:", Position =
{X_LABEL, Y1}, Size = {100, 20}, FontSize = 14, HTextAlign = "Right",
Color         =         {255,         255,         255}         })
--                Status                MODO                (VALOR)
layout["ENC1_Mode"] = { PrettyName = "Status ENCODER 1 - MODO",
Style = "Text", Position = {X_VALUE, Y1}, Size = {100, 20}, Color =
{128,            128,            128,            77}            }
Y1             =             Y1             +             SPACING

--               Status               ESTADO               (LABEL)
table.insert(graphics, { Type = "Label", Text = "ESTADO:", Position
= {X_LABEL, Y1}, Size = {100, 20}, FontSize = 14, HTextAlign =
"Right",      Color      =      {255,      255,      255}      })
--               Status               ESTADO               (VALOR)
layout["ENC1_State"] = { PrettyName = "Status ENCODER 1 - ESTADO",
Style = "Text", Position = {X_VALUE, Y1}, Size = {100, 20}, Color =
{128,            128,            128,            77}            }
Y1             =             Y1             +             SPACING
```

```
--                Status                HTTP                (LABEL)
table.insert(graphics, { Type = "Label", Text = "HTTP STATUS:",
Position = {X_LABEL, Y1}, Size = {100, 20}, FontSize = 14, HTextAlign
=     "Right",     Color     =     {255,     255,     255}     })
--                Status                HTTP                (VALOR)
layout["ENC1_HTTPStatus"] = {PrettyName = "Status ENCODER 1 - HTTP",
Style = "Text", Position = {X_VALUE, Y1}, Size = {100, 20}, Color =
{80,                  80,                  80}                  }


-------------------------------------------------------
--           CONTROLES           DO           ENCODER           2
-------------------------------------------------------
local    Y2    =    START_Y    +    GROUP_HEIGHT    +    10    +    25

--                        Botão                        START/STOP
layout["ENC2_StartButton"]                  =                  {
    PrettyName          =          "BOTAO          INICIAR/PARAR",
    Style                       =                       "Button",
    Position             =             {X_BUTTON,             Y2},
    Size                 =                {60,                20},
    Color           =           {160,           82,           45}
}

--                Status                MODO                (LABEL)
table.insert(graphics, { Type = "Label", Text = "MODO:", Position =
{X_LABEL, Y2}, Size = {100, 20}, FontSize = 14, HTextAlign = "Right",
Color         =         {255,         255,         255}         })
--                Status                MODO                (VALOR)
layout["ENC2_Mode"] = { PrettyName = "Status ENCODER 2 - MODO",
Style = "Text", Position = {X_VALUE, Y2}, Size = {100, 20}, Color =
{128,            128,            128,            77}            }
Y2             =             Y2             +             SPACING

--               Status               ESTADO               (LABEL)
table.insert(graphics, { Type = "Label", Text = "ESTADO:", Position
= {X_LABEL, Y2}, Size = {100, 20}, FontSize = 14, HTextAlign =
"Right",      Color      =      {255,      255,      255}      })
--               Status               ESTADO               (VALOR)
layout["ENC2_State"] = { PrettyName = "Status ENCODER 2 - ESTADO",
Style = "Text", Position = {X_VALUE, Y2}, Size = {100, 20}, Color =
{128,            128,            128,            77}            }
Y2             =             Y2             +             SPACING

--                Status                HTTP                (LABEL)
table.insert(graphics, { Type = "Label", Text = "HTTP STATUS:",
Position = {X_LABEL, Y2}, Size = {100, 20}, FontSize = 14, HTextAlign
=     "Right",     Color     =     {255,     255,     255}     })
--                Status                HTTP                (VALOR)
```

```
layout["ENC2_HTTPStatus"] = { PrettyName = "Status ENCODER 2 -
HTTP", Style = "Text", Position = {X_VALUE, Y2}, Size = {100, 20},
Color = {80, 80, 80} }

return layout, graphics


end

if Controls then --
========== -- 1.
PROPRIEDADES DE CONFIGURAÇÃO (Ajustado) --
========== local HOST_IP
= Properties["HostIP"].Value local MATROX_PORT = Properties["MatroxPort"].Value

local USERNAME = Properties["Username"].Value

local PASSWORD = Properties["Password"].Value --
========== -- 2. VARIÁVEIS
DE ESTADO E CONTROLES --
========== local
isRecording1 -- Estado do Encoder 1 local isRecording2 -- Estado do Encoder 2 (NOVO)
local isStreaming1 local isStreaming2 local json = require("json")

-- VARIÁVEIS GLOBAIS DE PISCA (Duplicadas)
local blinkingTimer1 = nil
local blinkingTimer2 = nil -- NOVO
local BLINK_INTERVAL = 0.5

-- Cores
local COLOR_IDLE_READY = "#A0522D"
local COLOR_REC_LIGHT = "#FF4500"
local COLOR_REC_DARK = "#8B0000"
local COLOR_STREAM_LIGHT = "#00BFFF" -- Azul Claro
local COLOR_STREAM_DARK = "#1E90FF" -- Azul Escuro

-- VARIÁVEL GLOBAL PARA PASSAR CONTEXTO DE COMANDO
_G.CurrentCommandType = "STATUS"
_G.RetryCount = 0
local MAX_RETRIES = Properties["MaxRetries"].Value
-- CONTROLES (Duplicados)
local StartButton1 = Controls.ENC1_StartButton
local TextFieldStatus1 = {Controls.ENC1_Mode, Controls.ENC1_State,
Controls.ENC1_HTTPStatus} -- Assume que é uma tabela de 3 campos
[Mode, State, HTTP]

local StartButton2 = Controls.ENC2_StartButton -- NOVO BOTÃO
local TextFieldStatus2 = {Controls.ENC2_Mode, Controls.ENC2_State,
Controls.ENC2_HTTPStatus} -- NOVO INDICADOR
```

```
--==========================================================
--               3.               FUNÇÕES               AUXILIARES
--==========================================================

--    Funções    de    controle    da    pisca    para    ENC1
local                    function                    stopBlinking1()
if                       blinkingTimer1                        then
    Timer.Clear(blinkingTimer1)
    blinkingTimer1                       =                      nil
end
end

local                   function                   startBlinking1()
    if (not isRecording1 and not isStreaming1) or not StartButton1
then
        stopBlinking1()
        if                     StartButton1                     then
            StartButton1.Color          =          COLOR_IDLE_READY
        end
        return
    end

    local  LIGHT_COLOR  =  isRecording1  and  COLOR_REC_LIGHT  or
COLOR_STREAM_LIGHT
    local  DARK_COLOR  =  isRecording1  and  COLOR_REC_DARK  or
COLOR_STREAM_DARK

    if      StartButton1.Color      ==      LIGHT_COLOR      then
        StartButton1.Color               =               DARK_COLOR
    else
        StartButton1.Color               =              LIGHT_COLOR
    end
    blinkingTimer1        =        Timer.CallAfter(startBlinking1,
BLINK_INTERVAL)
end

--  Funções  de  controle  da  pisca  para  ENC2  (Duplicadas)
local                    function                    stopBlinking2()
    if                     blinkingTimer2                       then
        Timer.Clear(blinkingTimer2)
        blinkingTimer2                     =                     nil
    end
end

local                   function                   startBlinking2()
    if (not isRecording2  and not isStreaming2) or not StartButton2
then
        stopBlinking2()
```

```
        if StartButton2 then
            StartButton2.Color = COLOR_IDLE_READY
        end
        return
    end

    local LIGHT_COLOR = isRecording2 and COLOR_REC_LIGHT or
COLOR_STREAM_LIGHT
    local DARK_COLOR = isRecording2 and COLOR_REC_DARK or
COLOR_STREAM_DARK

    if StartButton2.Color == LIGHT_COLOR then
        StartButton2.Color = DARK_COLOR
    else
        StartButton2.Color = LIGHT_COLOR
    end
    blinkingTimer2 = Timer.CallAfter(startBlinking2,
BLINK_INTERVAL)
end


-- Transforma os dados de status do encoder em uma tabela (MANTIDA)
function split_status(status_string)
    local results = {}
    for chunk in string.gmatch(status_string, '([^,]+)') do
        local clean_chunk = string.gsub(chunk, "^%s*(.-)%s*$",
"%1")
        table.insert(results, clean_chunk)
    end
    return results
end

-- Codificar estilos de ícone (MANTIDA)
local function getIconStyle(iconCode)
return json.encode{
    IconString = iconCode
}
end

local ICON_STOP = getIconStyle(string.char(0xef,0x87,0xaf))
local ICON_RECORD = getIconStyle(string.char(0xef,0x86,0xa4))
local ICON_REFRESH = getIconStyle(string.char(0xef,0x86,0xa5))
local ICON_ALERT = getIconStyle(string.char(0xef,0x84,0x81))
local ICON_RSS = getIconStyle(string.char(0xef,0x86,0xaa))

-- Funções de atualização unificadas
```

```
-- Funções de atualização unificadas local function updateEncoderStatus(enc_num, enc_mode, enc_state, wasRecordingOrStreaming, startButton, statusFields, stopBlinkFunc, startBlinkFunc)

local isRecordingCurrent = (enc_mode == "RECORD" and enc_state == "ON")
local isStreamingCurrent = ( (enc_mode == "RTMP" or enc_mode == "RTSP") and enc_state == "ON")

local isCurrentlyActive = isRecordingCurrent or isStreamingCurrent

-- ATUALIZA AS VARIÁVEIS GLOBAIS
if enc_num == 1 then
    isRecording1 = isRecordingCurrent
    isStreaming1 = isStreamingCurrent
else
    isRecording2 = isRecordingCurrent
    isStreaming2 = isStreamingCurrent
end

-- ... (Atualização das variáveis isRecordingX e isStreamingX)

local mode_display = enc_mode or "N/A"
local state_display = enc_state or "N/A"

-- DEFINE AS CORES COM BASE NO MODO ATIVO
local LIGHT_COLOR = isRecordingCurrent and COLOR_REC_LIGHT or isStreamingCurrent and COLOR_STREAM_LIGHT or COLOR_IDLE_READY
local IDLE_COLOR = COLOR_IDLE_READY

-- NOVO: Determinar a cor IDLE com base no modo
local IDLE_COLOR_FINAL = IDLE_COLOR -- Cor padrão (Recording Idle)
if (enc_mode == "RTMP" or enc_mode == "RTSP") and not isCurrentlyActive then
    IDLE_COLOR_FINAL = COLOR_STREAM_DARK -- Cor IDLE para Streaming
end

if isCurrentlyActive then
    -- ... (Lógica de ATIVO e Blinking: MANTIDA)
    if not wasRecordingOrStreaming then
        stopBlinkFunc()
        startButton.Color = LIGHT_COLOR
        startBlinkFunc()
    end
    startButton.Style = ICON_STOP
    startButton.String = "STOP"
else
    -- IDLE/READY
```

```
    if                  wasRecordingOrStreaming                 then
        stopBlinkFunc()
    end

    --   MUDE   ESTA   LINHA:   Use   a   cor   IDLE_COLOR_FINAL
    if   enc_mode   ==   "RTMP"   or   enc_mode   ==   "RTSP"   then
        startButton.Style                =                ICON_RSS
        startButton.String               =               "Stream"
    else
        startButton.Style               =               ICON_RECORD
        startButton.String                 =                 "REC"
    end

    startButton.Color              =              IDLE_COLOR_FINAL

end

statusFields[1].String               =               mode_display
statusFields[2].String               =               state_display


endfunction     handle_response(tbl,     code,     data,     err)
    local             responseText             =             data
    local        command_type        =        _G.CurrentCommandType
    local           isSuccessfulResponse          =          false

    local  COLOR_HTTP_OK  =  "#00FF00"          --  Verde  brilhante
    local  COLOR_HTTP_RETRY  =  "#FFD700"      --  Dourado/Amarelo
    local   COLOR_HTTP_ERROR   =   "#FF0000"         --   Vermelho

    -- TRATAMENTO DE CONECTIVIDADE (APLICA-SE AOS DOIS ENCODERS)
    if code == 200 and responseText ~= "RETRY" and responseText ~=
""                                                              then
        isSuccessfulResponse                =                 true
        _G.RetryCount                     =                     0
        TextFieldStatus1[3].String = "OK" -- HTTP Status para ENC1
        TextFieldStatus1[3].Color         =          COLOR_HTTP_OK
        TextFieldStatus2[3].String = "OK" -- NOVO: HTTP Status para
ENC2
        TextFieldStatus2[3].Color         =          COLOR_HTTP_OK

    else
        _G.RetryCount       =       _G.RetryCount       +       1
        Log.Error(string.format("Falha/Retry  Matrox.  Tentativas:
%i. Code=%i. Erro=%s", _G.RetryCount, code, err or responseText))
        local   retry_msg   =   string.format("RETRY...   (%i/%i)",
_G.RetryCount,                                          MAX_RETRIES)
        TextFieldStatus1[3].String          =           retry_msg
        TextFieldStatus1[3].Color        =        COLOR_HTTP_RETRY
```

```
        TextFieldStatus2[3].String    =    retry_msg    --    NOVO
        TextFieldStatus2[3].Color        =        COLOR_HTTP_RETRY

    end

    --              VERIFICAÇÃO              DE              TIMEOUT
    if        _G.RetryCount        >=        MAX_RETRIES        then
        local  msg  =  string.format("TIMEOUT!  Sem  resposta  há  %d
seg.",               MAX_RETRIES               *               2)
        TextFieldStatus1[3].String              =              msg
        TextFieldStatus1[3].Color        =        COLOR_HTTP_ERROR
        TextFieldStatus2[3].String      =      msg      --      NOVO
        TextFieldStatus2[3].Color        =        COLOR_HTTP_ERROR

        --     DESLIGA     TODOS     OS     CONTROLES     VISUAIS
        local          default_style          =          ICON_ALERT
        local          default_color          =          "#808080"
        local         default_text         =         "Desconhecido"

        StartButton1.Color             =             default_color
        StartButton1.Style             =             default_style
        TextFieldStatus1[1].String         =         default_text
        TextFieldStatus1[2].String         =         default_text
        stopBlinking1()    --    Para    a    pisca    do    ENC1

        StartButton2.Color             =             default_color
StartButton2.Style                 =                 default_style
        TextFieldStatus2[1].String         =         default_text
        TextFieldStatus2[2].String         =         default_text
        stopBlinking2()    --    Para    a    pisca    do    ENC2
        return
    end

    --       CASO       A       REQ.       HTTP       FALHE/RETRY
    if           not           isSuccessfulResponse           then
        StartButton1.Color               =               "#FFC000"
        StartButton1.Style             =             ICON_REFRESH
        stopBlinking1()

        StartButton2.Color               =               "#FFC000"
        StartButton2.Style             =             ICON_REFRESH
        stopBlinking2()
        return
    end

    -- LÓGICA DE CONTROLE (Mantida, pois só precisa forçar o status)
    if        command_type        ==        "CONTROL"        then
        if  string.find(responseText,  "SUCCESS",  1,  true)  then
            Log.Message("Comando  de  controle  Matrox  OK.  Forçando
```

```
atualização de status.")
            sendMatroxRequest("GetStatus", "STATUS")
        else
            Log.Error("Comando Matrox falhou. Resposta: " ..
responseText)
            TextFieldStatus1[3].String = "ERRO (Ver Log)"
            TextFieldStatus2[3].String = "ERRO (Ver Log)"
        end

--    LÓGICA    DE    STATUS    (Atualiza    ENC1    e    ENC2)
    elseif command_type == "STATUS" then

        -- Extrai o status do ENC1
        local enc1_status_full = string.match(responseText,
"ENC1:(.-),ENC2:")
        if not enc1_status_full then enc1_status_full =
string.match(responseText, "ENC1:(.*)") end
        local enc1_array =
split_status(string.gsub(enc1_status_full or "", "%s", ""))

        -- Extrai o status do ENC2
        local enc2_status_full = string.match(responseText,
"ENC2:(.-),NAME:") -- Assume que NAME: é o próximo campo
        if not enc2_status_full then enc2_status_full =
string.match(responseText, "ENC2:(.*)") end
        local enc2_array =
split_status(string.gsub(enc2_status_full or "", "%s", ""))

        -- ATUALIZA ENC1
        -- O quarto argumento deve ser o estado "anterior":
isRecording1 OU isStreaming1
        updateEncoderStatus(1, enc1_array[1], enc1_array[2],
isRecording1 or isStreaming1, StartButton1, TextFieldStatus1,
stopBlinking1, startBlinking1)

        -- ATUALIZA ENC2
        updateEncoderStatus(2, enc2_array[1], enc2_array[2],
isRecording2 or isStreaming2, StartButton2, TextFieldStatus2,
stopBlinking2, startBlinking2)
    end
end

--==========================================================
--       5.       FUNÇÃO       GENÉRICA       DE       REQUISIÇÃO
--==========================================================
function sendMatroxRequest(command, command_type)
    _G.CurrentCommandType = command_type
    local encoded_password = string.gsub(PASSWORD, "@", "%%40")
local HOST_IP = Properties["HostIP"].Value
```

```
    local                        URL_comando                        =
string.format("https://%s:%s@%s:%d/Monarch/syncconnect/sdk.aspx?co
mmand=%s",
                                    USERNAME,     encoded_password,
HOST_IP,                    MATROX_PORT,                   command)

    HttpClient.Get                                                  {
        Url                     =                     URL_comando,
        Timeout                         =                        5,
        EventHandler               =                handle_response
    }
end

--==============================================
--           6.           CALLBACKS           DOS           BOTÕES
--==============================================

--                          CALLBACK                          ENC1
StartButton1.EventHandler                =               function()
    if  isRecording1  or  isStreaming1  then  --  Verifica  ambos
        sendMatroxRequest("StopEncoder1",               "CONTROL")
    else
        sendMatroxRequest("StartEncoder1",              "CONTROL")
    end
end

--                          CALLBACK                          ENC2
StartButton2.EventHandler                =               function()
    if  isRecording2  or  isStreaming2  then  --  Verifica  ambos
        sendMatroxRequest("StopEncoder2",               "CONTROL")
    else
        sendMatroxRequest("StartEncoder2",              "CONTROL")
    end
end




--=====================================================
--       7.       CÓDIGO       PRINCIPAL       /       INICIALIZAÇÃO
--=====================================================

Log.Message("Script Matrox HDX v4.0 carregado. Iniciando Polling.")


--           ATUALIZAÇÃO           EM           TEMPO           REAL
local                     function                     startPolling()
    -- math.max garante que o intervalo nunca seja 0 (o que travaria
```

```
o                                                                   Core)
    local             interval             =             math.max(1,
Properties["PollingInterval"].Value)

    sendMatroxRequest("GetStatus",                        "STATUS")

    --    Correção:    Use    a    variável    'interval'    aqui
    Timer.CallAfter(startPolling,                         interval)
end


startPolling() Log.Message(string.format("Inicialização concluída. Polling ativo a cada
%s segundos.", Properties["PollingInterval"].Value)) end
```